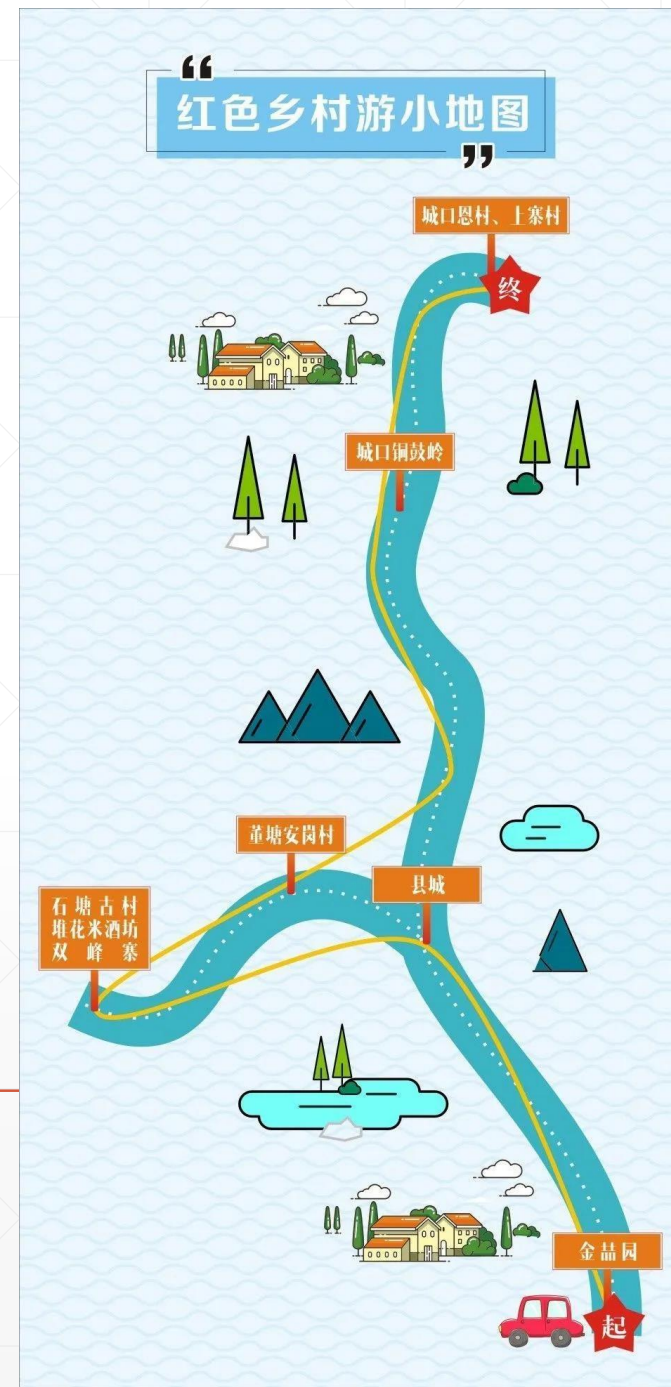


学用JDK多线程组件 之

使用CyclicBarrier

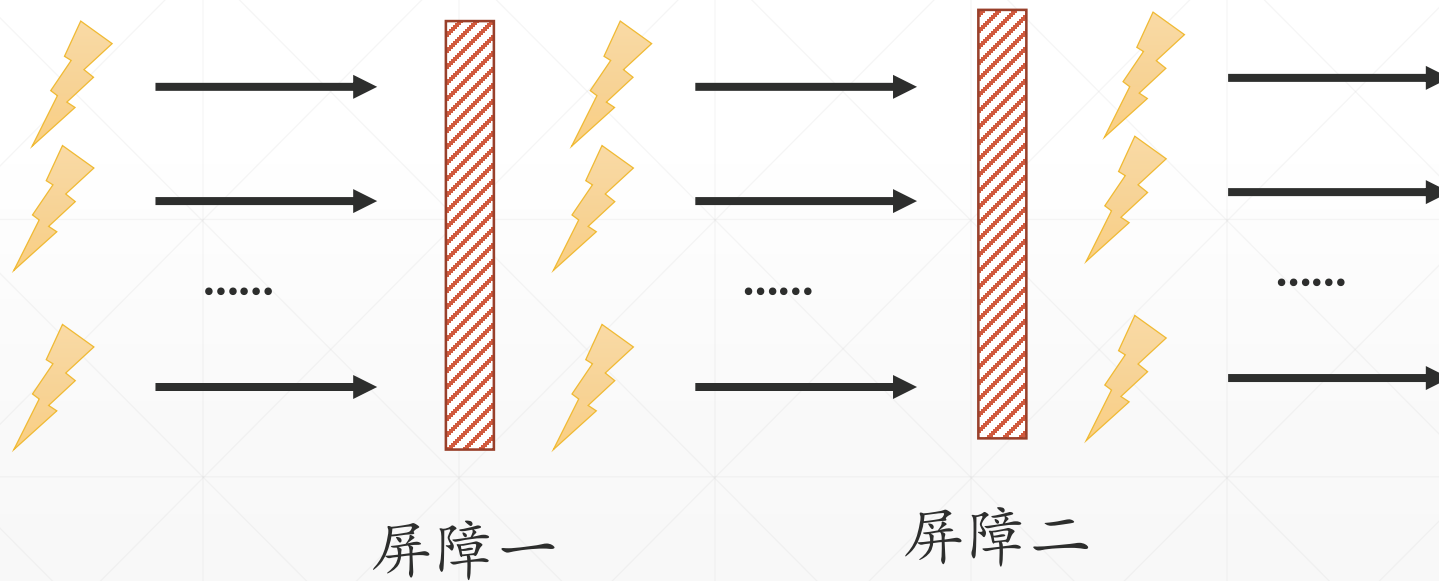
北京理工大学计算机学院
金旭亮



概述



CyclicBarrier组件适用于这种情况：你希望创建一组任务，它们并发地执行工作，直到所有的任务都完成，大家再一起进行下一步。



构造CyclicBarrier实例

```
public CyclicBarrier(int parties, Runnable barrierAction)
```

- ▶ `parties` 参数，表明有多少个线程需要同步。
- ▶ `barrierAction` 参数引用一个 `Runnable` 对象，当所有参与同步的线程都到了“屏障（即同步点）”时，将会回调此 `Runnable` 对象的 `run` 方法。

实现线程间的等待.....

```
public int await() throws InterruptedException, BrokenBarrierException {  
}
```

▶ 当线程调用CyclicBarrier的await()方法时，它会在此处阻塞等待，等到其他所有参与同步的线程也调用了await()方法（即也执行到了await()这句）后，所有线程会一起继续往下执行。

▶ 线程继续执行之前，会调用CyclicBarrier构造时传入的Runnable对象（如果有的话）的run()方法。

掌握CyclicBarrier的基本用法

```
Run: CyclicBarrierTest x
"C:\Program Files\Java\jdk-17.0.3.1\
Thread-1开始运行...
Thread-0开始运行...
Thread-0到达同步点。
Thread-1到达同步点。
所有线程都已经成功到达线程同步点
Thread-1运行结束。
Thread-0运行结束。
Process finished with exit code 0
```

```
private static void howToUseCyclicBarrier() {
    //指定有两个需要同步的线程，并且传入一个回调函数，当线程全部到达同步点后调用
    CyclicBarrier cb = new CyclicBarrier( parties: 2, () -> {
        System.out.println("所有线程都已经成功到达线程同步点");
    });
    for (int i = 0; i < 2; i++) {
        new Thread(() -> {
            var threadName :String = Thread.currentThread().getName();
            try {
                System.out.println(threadName + "开始运行...");
                //随机休眠一段时间
                Thread.sleep((long) (Math.random() * 10000));
                System.out.println(threadName + "到达同步点。");
                //等待其他线程的到达
                cb.await();
                System.out.println(threadName + "运行结束。");
            } catch (InterruptedException | BrokenBarrierException e) {
                throw new RuntimeException(e);
            }
        }).start();
    }
}
```

重用CyclicBarrier

相同的过程，重复两次

```
Run: CyclicBarrierTest x
"C:\Program Files\Java\jdk-17.0.3.1
Thread-3正在等待其它线程.....
Thread-1正在等待其它线程.....
Thread-4正在等待其它线程.....
Thread-4重新出发...
Thread-3重新出发...
Thread-1重新出发...
Thread-2正在等待其它线程.....
Thread-0正在等待其它线程.....
Thread-5正在等待其它线程.....
Thread-5重新出发...
Thread-2重新出发...
Thread-0重新出发...

Process finished with exit code 0
```

```
private static void reuseCyclicBarrier() {
    //每次有3个线程需要同步
    CyclicBarrier cb = new CyclicBarrier(parties: 3);
    //但现在有6个线程.....
    for (int i = 0; i < 6; i++) {
        Runnable threadFunc = () -> {
            var threadName : String = Thread.currentThread().getName();
            try {
                Thread.sleep((long) (Math.random() * 10000));
                System.out.println(threadName + "正在等待其它线程.....");
                //await()方法是可以被重复调用的
                cb.await();
                System.out.println(threadName + "重新出发...");
            } catch (InterruptedException | BrokenBarrierException e) {
                throw new RuntimeException(e);
            }
        };
        new Thread(threadFunc).start();
    }
}
```



动手实验

如果在构造CyclicBarrier时传入的参与线程个数，大于真正运行的调用了await()方法的线程个数，会发生什么情况？

请编写一个示例进行测试。



多线程、多阶段同步示例



示例启动三个线程，设置三个同步地点。

三个线程随机休眠特定的时间，等到三个线程都到达了同一个地点，再前往下一个地点。


```

private static void multiPhase() {
    //代表当前阶段的编号
    AtomicInteger currentPhase = new AtomicInteger( initialValue: 1);
    final CyclicBarrier cb = new CyclicBarrier( parties: 3, () -> {
        System.out.println("\n===阶段" + currentPhase.getAndAdd( delta: 1) + "完成===\n");
    }); //有3个参与者
    for (int i = 0; i < 3; i++) {
        Runnable runnable = () -> {
            try {
                var threadName :String = Thread.currentThread().getName();
                Thread.sleep((long) (Math.random() * 10000));
                arriveDestination( info: "线程" + threadName + "到达集合地点1.", cb);
                cb.await();
                Thread.sleep((long) (Math.random() * 10000));
                arriveDestination( info: "线程" + threadName + "到达集合地点2.", cb);
                cb.await();
                Thread.sleep((long) (Math.random() * 10000));
                arriveDestination( info: "线程" + threadName + "到达最后目的地.", cb);
                cb.await();
                System.out.println("线程" + Thread.currentThread().getName() + "工作结束");
            } catch (Exception e) {
                e.printStackTrace();
            }
        };
        Thread th = new Thread(runnable);
        th.start();
    }
}

```

示例启动了3个线程，每个随机休眠若干秒。

在线程函数中，调用3次await()方法，其实就是定义了3个“屏障”，3个线程在此同步，等到所有线程都干完了活，一起进入下一个阶段。

运行截图



```
private static void arriveDestination(String info, CyclicBarrier cb) {  
    StringBuilder sb = new StringBuilder();  
    //获取当前同步阶段中已经在等待的线程个数  
    int waitingNumber = cb.getNumberWaiting();  
    sb.append(info);  
    switch (waitingNumber) {  
        case 0 -> sb.append("我是第一个到的。");  
        case 1 -> sb.append("前面已有一人已到达。");  
        case 2 -> sb.append("现在人都到齐了。");  
    }  
    System.out.println(sb.toString());  
}
```

```
Run: CyclicBarrierTest x  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"  
线程Thread-1到达集合地点1。我是第一个到的。  
线程Thread-2到达集合地点1。前面已有一人已到达。  
线程Thread-0到达集合地点1。现在人都到齐了。  
  
===阶段1完成===  
  
线程Thread-2到达集合地点2。我是第一个到的。  
线程Thread-1到达集合地点2。前面已有一人已到达。  
线程Thread-0到达集合地点2。现在人都到齐了。  
  
===阶段2完成===  
  
线程Thread-0到达最后目的地。我是第一个到的。  
线程Thread-2到达最后目的地。前面已有一人已到达。  
线程Thread-1到达最后目的地。现在人都到齐了。  
  
===阶段3完成===  
  
线程Thread-1工作结束  
线程Thread-0工作结束  
线程Thread-2工作结束  
  
Process finished with exit code 0
```

CyclicBarrier vs. CountdownLatch



CyclicBarrier要等到预定数量的线程都到达了同步点才能继续执行，针对是线程，而CountDownLatch可看成是一个简单的计数器，当计数值为0时，它触发一个“时间到”事件。



CyclicBarrier是可以“重用”的，多个线程可以反复地调用await()方法，而CountDownLatch则是“一次性用品”，计数值减到0后，“无法再复位”。